

Java Site Survey Tool

For Telecom / Internet Service Providers

Complete Development Milestone Guide — From Scratch to Production

Spring Boot | React.js + Tailwind | MySQL | JWT | Docker | MapLibre | iText

Total Milestones	14 Milestones across 8 Weeks
Backend	Spring Boot (Java), Spring Security, JPA, Hibernate
Frontend	React.js + Tailwind CSS + MapLibre GL
Database	MySQL — all tables match DB diagram in PDF
Auth	JWT Access Token + Refresh Token
Storage	Local FS (dev) / MinIO or S3 (prod)
Reports	iText / JasperReports — PDF generation
Roles	ADMIN ENGINEER CUSTOMER

0. Role-Based Access Control — Full Breakdown

Every API endpoint is secured based on these three roles. Read this section before coding any milestone.

Role	Access Level	APIs / Modules Accessible	Restrictions
 ADMIN	Full System — Superuser	• Manage organizations, users, roles • CRUD on properties / buildings / floors / spaces • Upload, replace, delete floor plans • Create & version checklist templates • View ALL survey responses • Generate & download PDF reports • View & export audit logs • Manage RF scans (all properties) • Full equipment & cable-path management • Assign engineers to properties	<i>None — full access to every module</i>
 ENGINEER	Survey & Field Operations	• View ONLY assigned properties • Upload floor plans (assigned site only) • Label / tag spaces on canvas • Submit & edit checklist responses • Upload attachments (photos / notes) • Add equipment & cable paths • Import / export RF scan files • View survey dashboards (own data) • Draft & save survey reports (no publish)	<i>Cannot access other orgs, cannot delete templates, cannot publish reports</i>
 CUSTOMER	Read-Only Viewer	• View OWN property only	<i>Zero write access — view only; no admin or engineering tools</i>

		• View floor plans (no edit) • View submitted checklist statuses • Download finalized PDF reports • View survey completion dashboards • View RF heatmap overlays	
--	--	--	--

DB Schema Reference (matches PDF DB Diagram exactly)

All 14 tables below are defined exactly as shown in the dbdiagram.io diagram in the PDF. Use this as your single source of truth when writing JPA entities.

1.1 organizations

organizations		
Column	Type	Notes
id	BIGINT	 PK
name	VARCHAR(200)	org display name
created_at	DATETIME	
updated_at	DATETIME	

1.2 users

users		
Column	Type	Notes
id	BIGINT	 PK
email	VARCHAR(320)	unique, login credential
full_name	VARCHAR(300)	
auth_provider	VARCHAR(100)	local / google / etc.
created_at	DATETIME	
updated_at	DATETIME	

1.3 memberships

memberships		
Column	Type	Notes
id	BIGINT	 PK
organization_id	BIGINT	 FK → organizations.id
user_id	BIGINT	 FK → users.id
role	role_type ENUM	ADMIN ENGINEER CUSTOMER
created_at	DATETIME	
updated_at	DATETIME	

1.4 properties

properties		
Column	Type	Notes

id	BIGINT	🔑 PK
organization_id	BIGINT	🔗 FK → organizations.id
name	VARCHAR(200)	
address_line1	VARCHAR(200)	
address_line2	VARCHAR(200)	
city	VARCHAR(100)	
state	VARCHAR(100)	
postal_code	VARCHAR(40)	
country	VARCHAR(100)	
boundary_type	geometry_type ENUM	
boundary_wkt	TEXT	WKT polygon of site boundary
centroid_lat	DECIMAL(10,7)	
centroid_lon	DECIMAL(10,7)	
created_at	DATETIME	
updated_at	DATETIME	

1.5 buildings

buildings		
Column	Type	Notes
id	BIGINT	🔑 PK
property_id	BIGINT	🔗 FK → properties.id
name	VARCHAR(200)	
code	VARCHAR(50)	
footprint_type	geometry_type ENUM	
footprint_wkt	TEXT	WKT polygon of building footprint
floors_count	INT	
created_at	DATETIME	
updated_at	DATETIME	

1.6 floors

floors		
Column	Type	Notes
id	BIGINT	🔑 PK
building_id	BIGINT	🔗 FK → buildings.id
level_label	VARCHAR(50)	e.g. G, 1, 2, B1
elevation_m	DECIMAL(8,2)	

plan_file_id	BIGINT	 FK → files.id
scale_ratio	VARCHAR(50)	e.g. 1:100
anchor_points	JSON	GeoJSON anchor coords
created_at	DATETIME	
updated_at	DATETIME	

1.7 spaces

spaces		
Column	Type	Notes
id	BIGINT	 PK
floor_id	BIGINT	 FK → floors.id
name	VARCHAR(200)	
type	space_type ENUM	
geometry_type	geometry_type ENUM	
geometry_wkt	TEXT	WKT polygon of space on floor plan
area_sq_m	DECIMAL(10,2)	
elevation_m	DECIMAL(8,2)	
notes	TEXT	
created_at	DATETIME	
updated_at	DATETIME	

1.8 files

files		
Column	Type	Notes
id	BIGINT	 PK
owner_type	VARCHAR(50)	polymorphic owner type
owner_id	BIGINT	polymorphic owner id
filename	VARCHAR(255)	
content_type	VARCHAR(120)	MIME type
storage_key	VARCHAR(512)	path or S3/MinIO key
size_bytes	BIGINT	
checksum_sha256	CHAR(64)	
created_at	DATETIME	
updated_at	DATETIME	

1.9 attachments

attachments		
Column	Type	Notes
id	BIGINT	 PK
owner_type	VARCHAR(50)	polymorphic owner type
owner_id	BIGINT	polymorphic owner id
file_id	BIGINT	 FK → files.id
tags	VARCHAR(200)	
metadata	JSON	
created_at	DATETIME	
updated_at	DATETIME	

1.10 equipment

equipment		
Column	Type	Notes
id	BIGINT	 PK
space_id	BIGINT	 FK → spaces.id
type	equipment_type ENUM	
model	VARCHAR(200)	
vendor	VARCHAR(200)	
power_watts	DECIMAL(10,2)	
heat_load_btuh	DECIMAL(10,2)	
mounting	VARCHAR(100)	
geometry_type	geometry_type ENUM	
geometry_wkt	TEXT	
serial_number	VARCHAR(120)	
created_at	DATETIME	
updated_at	DATETIME	

1.11 cable_paths

cable_paths		
Column	Type	Notes
id	BIGINT	 PK
property_id	BIGINT	 FK → properties.id
from_space_id	BIGINT	 FK → spaces.id
to_space_id	BIGINT	 FK → spaces.id
medium	cable_medium ENUM	copper fiber coax

length_m	DECIMAL(10,2)	
slack_loops	INT	
splice_points	INT	
geometry_type	geometry_type ENUM	
geometry_wkt	TEXT	
notes	TEXT	
created_at	DATETIME	
updated_at	DATETIME	

1.12 splice_points

splice_points		
Column	Type	Notes
id	BIGINT	 PK
cable_path_id	BIGINT	 FK → cable_paths.id
geometry_type	geometry_type ENUM	
geometry_wkt	TEXT	
enclosure_id	BIGINT	optional enclosure ref
notes	TEXT	
created_at	DATETIME	
updated_at	DATETIME	

1.13 checklist_templates

checklist_templates		
Column	Type	Notes
id	BIGINT	 PK
organization_id	BIGINT	 FK → organizations.id
name	VARCHAR(200)	
scope	VARCHAR(50)	space floor building property
version	INT	starts at 1, auto-increment on edit
schema_json	JSON	question definitions array
is_active	BOOLEAN	
created_at	DATETIME	
updated_at	DATETIME	

1.14 checklist_responses

checklist_responses		
Column	Type	Notes
id	BIGINT	 PK
template_id	BIGINT	 FK → checklist_templates.id
target_type	VARCHAR(50)	polymorphic: space floor building
target_id	BIGINT	polymorphic target id
answers_json	JSON	key-value answers map
photos_manifest	JSON	array of file_ids
submitted_by	BIGINT	 FK → users.id
submitted_at	DATETIME	
created_at	DATETIME	
updated_at	DATETIME	

1.15 rf_scans

rf_scans		
Column	Type	Notes
id	BIGINT	 PK
property_id	BIGINT	 FK → properties.id
floor_id	BIGINT	 FK → floors.id
tool	rf_tool ENUM	VISTUMBLER KISMET SPLAT
raw_file_id	BIGINT	 FK → files.id
parsed_json	JSON	
heatmap_file_id	BIGINT	 FK → files.id
notes	TEXT	
created_at	DATETIME	
updated_at	DATETIME	

1.16 reports

reports		
Column	Type	Notes
id	BIGINT	 PK
property_id	BIGINT	 FK → properties.id
requested_by	BIGINT	 FK → users.id
parameters	JSON	report config options
pdf_file_id	BIGINT	 FK → files.id
status	VARCHAR(30)	PENDING GENERATING DONE FAILED

created_at	DATETIME	
updated_at	DATETIME	

1.17 audit_logs

audit_logs		
Column	Type	Notes
id	BIGINT	 PK
actor_id	BIGINT	 FK → users.id
action	VARCHAR(100)	CREATE UPDATE DELETE LOGIN etc.
entity_type	VARCHAR(50)	
entity_id	BIGINT	
change_set	JSON	before/after values
created_at	DATETIME	

Development Milestones — Full Detail

Milestone 1 — Core Setup: Environment, DB & Project Skeleton (Week 1)

 **GOAL:** Bootstrap the entire project so every developer can run it locally in one command.

 **TECH STACK:** Spring Boot 3.x • Maven • MySQL 8 • Docker Compose • GitHub Actions

1. GitHub Repository

- Create repo with branches: main, develop, feature/*, hotfix/*
- Add .gitignore for Java/Maven, .env files
- Write README with setup instructions

2. Spring Boot Project Init

- Use start.spring.io: Web, JPA, MySQL Driver, Validation, Lombok, Security, DevTools
- application.yml — configure:
 - spring.datasource (host, port, schema, user, password)
 - spring.jpa.hibernate.ddl-auto = validate (never auto-drop in prod)
 - server.port = 8080
- Create base packages: controller, service, repository, entity, dto, config, exception

3. MySQL Schema — ALL 17 Tables

- Create schema: site_survey_db
- Write V1__init.sql Flyway migration containing ALL tables from the DB diagram:
 - organizations, users, memberships
 - properties, buildings, floors, spaces
 - files, attachments
 - equipment, cable_paths, splice_points
 - checklist_templates, checklist_responses
 - rf_scans, reports, audit_logs
- Create all ENUM types: role_type, space_type, geometry_type, equipment_type, cable_medium, rf_tool
- Add all FK constraints, NOT NULL constraints, and indexes on FK columns

4. Docker Compose

- Service: mysql — image mysql:8, expose 3306, volume for data persistence
- Service: app — build from Dockerfile, depends_on: mysql, env_file: .env
- docker compose up --build should spin everything up

5. CI/CD Pipeline (GitHub Actions)

- On push to develop: run mvn test
- On push to main: run mvn package, push Docker image to registry

✓ **OUTCOME:** docker compose up starts MySQL + Spring Boot. Schema is created with all 17 tables. App starts on :8080 with no errors.

Milestone 2 — Authentication, JWT & Role-Based Access (Week 2)

 **GOAL:** Secure every endpoint using JWT tokens. Implement login, register, refresh, and role guards.

 **TECH STACK:** Spring Security • JWT (jjwt library) • BCrypt • Refresh Token • @PreAuthorize

1. Entities — users + memberships

- User.java — map to users table (id, email, fullName, authProvider, createdAt, updatedAt)
- Membership.java — map to memberships table (id, organizationId, userId, role enum, createdAt)
- RoleType.java — Java enum: ADMIN, ENGINEER, CUSTOMER
- UserRepository, MembershipRepository — JPA interfaces

2. Security Config

- SecurityFilterChain — permit /api/auth/**, block everything else
- JwtAuthFilter extends OncePerRequestFilter — validate Bearer token per request
- JwtUtil — generateAccessToken (15 min), generateRefreshToken (7 days), validateToken, extractEmail
- PasswordEncoder bean — BCryptPasswordEncoder

3. Auth APIs

- POST /api/auth/register — { email, password, fullName } → 201 Created
- POST /api/auth/login — { email, password } → { accessToken, refreshToken, expiresIn }
- POST /api/auth/refresh — { refreshToken } → { accessToken, refreshToken }
- POST /api/auth/logout — invalidate refresh token (store invalidated tokens in DB or cache)

4. Role Guards

- @PreAuthorize("hasRole('ADMIN')") on admin-only endpoints
- @PreAuthorize("hasAnyRole('ADMIN','ENGINEER')") on field operation endpoints
- @PreAuthorize("isAuthenticated()") on customer read-only endpoints
- Custom AccessDeniedHandler and AuthenticationEntryPoint — return clean JSON errors

5. User Management APIs (Admin only)

- POST /api/admin/users — create user with role
- GET /api/admin/users — list all users (paginated)
- GET /api/users/me — get own profile (any role)
- PUT /api/admin/users/{id}/role — change role

✅ **OUTCOME:** Login returns JWT. Calling `/api/admin/*` without ADMIN role returns 403. Token refresh works. All 3 roles can authenticate.

Milestone 3 — Organization & User Membership Management (Week 2)

 **GOAL:** Manage multi-tenant organizations and assign users to orgs with specific roles via memberships table.

 **TECH STACK:** Spring Boot REST • JPA • MySQL — organizations + memberships tables

1. Organization Entity

- Organization.java — id, name, createdAt, updatedAt
- @OneToMany relationship to Membership
- OrganizationRepository, OrganizationService, OrganizationController

2. Membership Entity

- Membership.java — id, organizationId, userId, role (RoleType enum)
- Composite unique constraint: (organization_id, user_id)
- MembershipRepository: findByOrganizationId, findByUserId

3. Organization APIs (Admin)

- POST /api/organizations — create org
- GET /api/organizations — list orgs (super-admin or own org only)
- GET /api/organizations/{id} — get org details
- PUT /api/organizations/{id} — update name
- DELETE /api/organizations/{id} — soft delete

4. Membership APIs (Admin)

- POST /api/organizations/{orgId}/members — add user to org with role
- GET /api/organizations/{orgId}/members — list members
- PUT /api/organizations/{orgId}/members/{userId} — change member role
- DELETE /api/organizations/{orgId}/members/{userId} — remove member

 **OUTCOME:** Orgs are created. Users are assigned to orgs with roles via memberships table. Role lookup for auth works through org membership.

Milestone 4 — Property Hierarchy: Property→Building→Floor→Space (Week 3)

 **GOAL:** Implement the full physical site hierarchy exactly as modeled in the DB diagram.

 **TECH STACK:** Spring Boot REST • JPA @OneToMany • MySQL — properties, buildings, floors, spaces

1. Property Entity

- Property.java — all fields from DB diagram: id, organizationId, name, address lines, city, state, postalCode, country
- boundaryType (GeometryType enum), boundaryWkt (TEXT), centroidLat, centroidLon
- @ManyToOne Organization, @OneToMany List<Building>

2. Building Entity

- Building.java — id, propertyId, name, code, footprintType (GeometryType), footprintWkt (TEXT), floorsCount
- @ManyToOne Property, @OneToMany List<Floor>

3. Floor Entity

- Floor.java — id, buildingId, levelLabel, elevationM, planFileId, scaleRatio, anchorPoints (JSON)
- @ManyToOne Building, @OneToMany List<Space>

4. Space Entity

- Space.java — id, floorId, name, type (SpaceType), geometryType, geometryWkt, areaSqM, elevationM, notes
- @ManyToOne Floor

5. Admin APIs — Full CRUD

- POST /api/properties — create property (Admin)
- GET /api/properties — list properties for org (Admin/Engineer)
- GET /api/properties/{id} — get property detail
- PUT /api/properties/{id} — update property (Admin)
- DELETE /api/properties/{id} — delete (Admin)
- POST /api/properties/{id}/buildings — add building
- GET /api/properties/{id}/buildings — list buildings
- POST /api/buildings/{id}/floors — add floor
- GET /api/buildings/{id}/floors — list floors
- POST /api/floors/{id}/spaces — add space
- GET /api/floors/{id}/spaces — list spaces

6. Role Visibility Rules

- ADMIN — sees all properties in their org
- ENGINEER — sees only properties they are assigned to (via memberships or assignment table)
- CUSTOMER — sees only their own property

✓ **OUTCOME: Full hierarchy Property→Building→Floor→Space is stored and queryable. Role-filtered list APIs work correctly.**

Milestone 5 — Floor Plan Upload & File Storage (Week 4)

 **GOAL:** Upload floor plan images/PDFs, persist files with full metadata in the files table, link to floors.

 **TECH STACK:** Spring Boot Multipart • Local FS / MinIO • SHA-256 checksum • files table

1. File Entity

- File.java — id, ownerType, ownerId, filename, contentType, storageKey, sizeBytes, checksumSha256
- ownerType / ownerId — polymorphic: can link to floor, checklist_response, rf_scan, etc.

2. Storage Service

- FileStorageService interface with two implementations:
 - LocalFileStorageService — saves to /uploads/{ownerType}/{ownerId}/{uuid}_{filename}
 - MinIOFileStorageService — uploads to configured bucket with storageKey
- Toggle via application.yml: storage.provider = local | minio

3. Upload API

- POST /api/files/upload — multipart/form-data, fields: file, ownerType, ownerId
- Validate MIME type (image/png, image/jpeg, image/webp, application/pdf)
- Validate max size (20 MB)
- Compute SHA-256 checksum before saving
- Save file, persist File entity, return file metadata JSON

4. Floor Plan Link

- After upload, call PUT /api/floors/{id}/plan-file with { fileId }
- This sets floors.plan_file_id = fileId
- GET /api/floors/{id}/plan-file — returns pre-signed URL or stream

5. Role Access

- ADMIN: Upload/delete any floor plan
- ENGINEER: Upload for assigned property only; cannot delete
- CUSTOMER: Download/stream only

 **OUTCOME:** Floor plan PDFs/images upload, are stored on disk/MinIO, metadata is in files table, and linked to the floor row.

Milestone 6 — Bulk Import: CSV / XLSX Parser (Week 4)

 **GOAL:** Allow bulk creation of spaces and buildings from spreadsheet files with preview + confirm workflow.

 **TECH STACK:** Apache POI (XLSX) • OpenCSV • Spring Boot • Preview-before-commit pattern

1. Import Endpoint

- POST /api/import/spaces — upload CSV or XLSX with columns matching spaces table
- POST /api/import/buildings — upload CSV or XLSX for buildings

2. Parse & Validate

- Detect file type by content-type or extension
- Parse rows into ImportSpaceDto / ImportBuildingDto
- Validate: required fields present, numeric types correct, enum values valid
- Collect errors per row: { rowNum, field, message }

3. Preview & Confirm Flow

- Step 1 — POST /api/import/spaces/preview → returns { valid: [...], errors: [...] }
- Step 2 — User reviews data in frontend (show table of parsed rows + errors)
- Step 3 — POST /api/import/spaces/confirm with session token from preview → persists valid rows
- Step 4 — POST /api/import/spaces/cancel → discards parsed data

4. Role Access

- ADMIN — import for any property
- ENGINEER — import for assigned property only
- CUSTOMER — no import access (403)

 **OUTCOME:** Engineer uploads a 50-row CSV of spaces. Preview shows parsed data + validation errors. Confirm saves valid rows to DB.

Milestone 7 — Canvas Labeling & Space Geometry (Week 5)

 **GOAL:** Engineers draw/click spaces on the floor plan canvas; geometry saved as WKT in `spaces.geometry_wkt`.

 **TECH STACK:** React + MapLibre GL • GeoJSON • WKT conversion • PUT `/api/spaces/{id}/geometry`

1. Frontend Canvas

- Load floor plan image as MapLibre raster layer
- Draw tool: polygon, rectangle, point — each saved to a space
- Display existing labeled spaces as colored polygons
- Click space → show name, type, area

2. Geometry API

- PUT `/api/spaces/{id}/geometry` — body: { `geometryType`, `geometryWkt`, `areaSqM` }
- Validate WKT string server-side
- Update `spaces.geometry_wkt`, `geometry_type`, `area_sq_m`
- GET `/api/floors/{id}/spaces/geometry` — return all spaces with WKT for canvas render

3. Role Access

- ADMIN & ENGINEER — can draw and save geometry
- CUSTOMER — can view labeled floor plan (read-only canvas)

 **OUTCOME:** Engineers open floor plan, draw polygons around rooms, save geometry — spaces are labeled and visible to all roles.

Milestone 8 — Checklist Templates & Survey Responses (Weeks 5-6)

 **GOAL:** Admins design checklist question schemas; engineers fill them in per space/floor/building.

 **TECH STACK:** Spring Boot • MySQL — checklist_templates + checklist_responses • JSON schema • multipart attachments

1. Checklist Template

- ChecklistTemplate.java — id, organizationId, name, scope, version, schemaJson, isActive
- schemaJson structure: array of { fieldId, label, type (text|number|boolean|select), required, options[] }
- Versioning: editing active template creates a new version; old version set isActive=false

2. Template APIs (Admin)

- POST /api/checklist-templates — create template
- GET /api/checklist-templates — list active templates for org
- GET /api/checklist-templates/{id} — get template + schema
- PUT /api/checklist-templates/{id} — update (creates new version)
- DELETE /api/checklist-templates/{id} — deactivate

3. Checklist Response

- ChecklistResponse.java — id, templateId, targetType, targetId, answersJson, photosManifest, submittedBy, submittedAt
- answersJson: { fieldId: value } map
- photosManifest: [fileId, ...] — links to uploaded photos in files table
- Draft support: submittedAt is null until engineer confirms submission

4. Response APIs (Engineer/Admin)

- POST /api/checklist-responses — save draft or submit response
- GET /api/checklist-responses?targetType=space&targetId=42 — get responses for a space
- PUT /api/checklist-responses/{id} — update draft
- POST /api/checklist-responses/{id}/submit — finalize submission (sets submittedAt)
- POST /api/checklist-responses/{id}/photos — attach photo file ids

5. Role Access

- ADMIN — create/edit templates; view all responses
- ENGINEER — submit and edit own responses; attach photos
- CUSTOMER — view submitted responses (no edit)

 **OUTCOME:** Admin creates a template with 10 questions. Engineer fills it for each space, attaches photos, and submits. Admin sees all submissions.

Milestone 9 — Equipment & Cable Path Management (Week 6)

 **GOAL:** Track all network equipment installed per space and cable routes between spaces, including splice points.

 **TECH STACK:** Spring Boot REST • JPA • MySQL — equipment, cable_paths, splice_points

1. Equipment Entity

- Equipment.java — id, spaceId, type (EquipmentType enum), model, vendor
- powerWatts, heatLoadBtuh, mounting, geometryType, geometryWkt, serialNumber
- EquipmentType enum: ACCESS_POINT, SWITCH, ROUTER, ONT, ANTENNA, CABINET, UPS, OTHER

2. Equipment APIs

- POST /api/spaces/{id}/equipment — add equipment to a space (Admin/Engineer)
- GET /api/spaces/{id}/equipment — list equipment in space
- GET /api/equipment/{id} — get single item
- PUT /api/equipment/{id} — update details (Admin/Engineer)
- DELETE /api/equipment/{id} — remove (Admin only)

3. Cable Path Entity

- CablePath.java — id, propertyId, fromSpaceId, toSpaceId, medium (CableMedium enum)
- lengthM, slackLoops, splicePoints, geometryType, geometryWkt, notes
- CableMedium enum: COPPER, FIBER, COAX, POWER

4. Cable Path APIs

- POST /api/cable-paths — create cable path (Admin/Engineer)
- GET /api/properties/{id}/cable-paths — list all cable paths for a property
- PUT /api/cable-paths/{id} — update path
- DELETE /api/cable-paths/{id} — delete (Admin only)

5. Splice Point Entity

- SplicePoint.java — id, cablePathId, geometryType, geometryWkt, enclosureId, notes
- POST /api/cable-paths/{id}/splice-points — add splice point
- GET /api/cable-paths/{id}/splice-points — list splice points

 **OUTCOME:** Full equipment inventory per space is tracked. Cable routes with length, medium, and splice points are stored and queryable.

Milestone 10 — Attachments & File Linking (Week 6)

 **GOAL:** Link uploaded files (photos, PDFs, notes) to any entity using the polymorphic attachments table.

 **TECH STACK:** Spring Boot • MySQL — attachments + files tables • Polymorphic associations

1. Attachment Entity

- Attachment.java — id, ownerType (String), ownerId (Long), fileId, tags, metadata (JSON)
- ownerType can be: space, equipment, cable_path, checklist_response, building, floor
- @ManyToOne File (join on fileId)

2. Attachment APIs

- POST /api/attachments — attach a file to any entity
 - Body: { ownerType, ownerId, fileId, tags, metadata }
- GET /api/attachments?ownerType=X&ownerId=Y — list attachments for entity
- GET /api/attachments/{id} — get single attachment with file URL
- DELETE /api/attachments/{id} — detach (does NOT delete the file)

3. Combined Upload + Attach Flow

- POST /api/files/upload → get fileId
- POST /api/attachments with { ownerType:'space', ownerId:5, fileId:returned_id }
- OR: single-shot endpoint POST /api/spaces/{id}/attachments (multipart) — upload + attach

 **OUTCOME:** Photos, PDFs, and files are attached to spaces/equipment/cables with tags. Polymorphic ownerType allows clean linking.

Milestone 11 — RF Scan Integration (Week 7)



GOAL: Import RF coverage data from Vistumbler, Kismet, and SPLAT! tools. Parse and store in `rf_scans` table. Overlay heatmap on floor plan canvas.



TECH STACK: Spring Boot • Apache Commons CSV / custom parsers • `rf_scans` table • MapLibre heatmap layer

1. RF Scan Entity

- `RfScan.java` — `id`, `propertyId`, `floorId`, `tool` (`RfTool` enum), `rawFileId`, `parsedJson`, `heatmapFileId`, `notes`
- `RfTool` enum: `VISTUMBLER`, `KISMET`, `SPLAT`
- `parsedJson` — normalized array of `{ lat, lon, signal_dbm, ssid, mac }` records

2. Upload & Parse APIs

- `POST /api/rf-scans/upload` — multipart: file + `{ propertyId, floorId, tool }`
 - Saves raw file → `files` table, creates `rf_scan` row with `rawFileId`
- `POST /api/rf-scans/{id}/process` — trigger parser for the tool type
 - Vistumbler: parse CSV export format
 - Kismet: parse Kismet `.kismet` or `.csv` log
 - SPLAT!: parse `.qth` and `.lrp` output files
 - Stores normalized array in `parsedJson`
- `POST /api/rf-scans/{id}/heatmap` — generate heatmap PNG from `parsedJson`, store in `files` table

3. Query APIs

- `GET /api/rf-scans?propertyId=X&floorId=Y` — list RF scans
- `GET /api/rf-scans/{id}` — get scan + parsed data
- `GET /api/rf-scans/{id}/heatmap` — stream heatmap image

4. Canvas Overlay (Frontend)

- Load heatmap PNG as semi-transparent MapLibre image layer over floor plan
- Toggle overlay on/off per scan

5. Role Access

- ADMIN & ENGINEER — upload, process, view all RF scans for assigned properties
- CUSTOMER — view heatmap overlay only (read-only)



OUTCOME: RF data from Vistumbler/Kismet/SPLAT! is imported, parsed, and displayed as a signal heatmap overlay on the floor plan.

Milestone 12 — Dashboards & PDF Report Generation (Week 7)

 **GOAL:** Build summary dashboards for survey progress and generate professional PDF reports using iText/JasperReports.

 **TECH STACK:** Spring Boot • iText 7 / JasperReports • MySQL aggregation queries • reports + files tables

1. Dashboard APIs

- GET /api/dashboard/survey-completion — % spaces with at least one submitted checklist response, per property
- GET /api/dashboard/checklist-status — count of open vs submitted responses, grouped by template
- GET /api/dashboard/equipment-count — equipment count per building/space
- GET /api/dashboard/rf-scan-coverage — floors with vs without RF scans per property
- GET /api/dashboard/properties-overview — summary card per property (spaces, surveys, equipment, cables)

2. Report Entity

- Report.java — id, propertyId, requestedBy, parameters (JSON), pdfFileId, status
- status enum: PENDING, GENERATING, DONE, FAILED
- Async generation: return reportId immediately, generate in background thread

3. Report Generation

- POST /api/reports/generate — { propertyId, includeFloorPlans, includeChecklists, includeEquipment, includeRf }
 - Creates Report row (PENDING), queues async job
- PDF contents:
 - Cover page: property name, org, date, requested by
 - Site overview: address, boundary map, building list
 - Floor plans: each floor plan image
 - Space inventory: table of spaces per floor with type, area
 - Equipment list: per space — type, model, vendor, serial
 - Cable paths: table of routes with medium and length
 - Checklist summary: responses per space with completion %
 - RF coverage: heatmap images per floor

4. Report APIs

- GET /api/reports — list reports for property
- GET /api/reports/{id} — get report status + metadata
- GET /api/reports/{id}/download — stream PDF (when status=DONE)

- DELETE /api/reports/{id} — delete report (Admin only)

5. Role Access

- ADMIN — generate and download reports for any property
- ENGINEER — request generation, cannot download until Admin approves
- CUSTOMER — download finalized reports for their property

✓ **OUTCOME:** Admin generates a PDF report for a property. Report includes floor plans, equipment table, checklist summary, and RF heatmaps as a downloadable file.

Milestone 13 — Audit Logs, Permissions Hardening & Testing (Week 8)

 **GOAL:** Log every state-changing action. Harden role guards. Write unit + integration tests for all layers.

 **TECH STACK:** Spring AOP • audit_logs table • JUnit 5 • Mockito • TestContainers • Spring Boot Test

1. Audit Log Entity

- AuditLog.java — id, actorId, action, entityType, entityId, changeSet (JSON), createdAt
- changeSet stores: { before: {...}, after: {...} } for UPDATE; null for CREATE; entity snapshot for DELETE

2. AuditService — AOP-based

- @Aspect AuditAspect — intercepts @Auditable annotated service methods
- Extract actorId from SecurityContext
- Build AuditLog and save asynchronously
- Audit all: CREATE, UPDATE, DELETE on properties, buildings, floors, spaces, equipment, cable_paths, checklist_responses, reports

3. Audit APIs (Admin only)

- GET /api/audit-logs — paginated list (Admin only)
- GET /api/audit-logs?entityType=X&entityId=Y — filter by entity
- GET /api/audit-logs?actorId=X — filter by user
- GET /api/audit-logs?fromDate=X&toDate=Y — date range filter

4. Permissions Hardening

- Review every endpoint — ensure @PreAuthorize is present
- Ensure engineer can ONLY read/write their assigned property (service-level check)
- Ensure customer token cannot access any write endpoint
- Add method-level security test for each role x endpoint combination

5. Unit Tests

- AuthService — register, login, refresh, invalid token
- PropertyService — CRUD, role filter
- FileStorageService — upload, checksum, retrieve
- ChecklistService — template versioning, response submission
- ReportService — async generation, status transitions

6. Integration Tests

- Use TestContainers for MySQL — spin up real DB for tests
- Full flow test: register → login → create property → upload floor plan → submit checklist → generate report
- Role violation tests: assert 403 for each disallowed role x action combination

 **OUTCOME:** Every state-change is logged in audit_logs. All 403 scenarios tested. Test coverage > 80% for service layer.

Milestone 14 — UAT, Staging Deployment & Documentation (Week 8)

 **GOAL:** Deploy to staging, run user acceptance testing with real ISP data, finalize all documentation.

 **TECH STACK:** Docker Compose / K8s • Swagger / OpenAPI 3 • GitHub Pages / Confluence • Release Notes

1. Staging Deployment

- Update Docker Compose for staging: add Nginx reverse proxy, MinIO container
- Configure environment-specific application-staging.yml (real DB creds, S3 keys)
- Deploy to staging server (VPS / cloud VM)
- Run Flyway migrations against staging DB
- Smoke test: login, upload floor plan, submit checklist, generate report

2. User Acceptance Testing

- Admin scenario: create org → add users → create property hierarchy → assign engineers
- Engineer scenario: login → view assigned property → upload floor plan → label spaces → submit checklists → log equipment
- Customer scenario: login → view property → view floor plan → download report
- RF scenario: engineer uploads Kismet scan → process → view heatmap on canvas
- Log all bugs found in GitHub Issues; fix critical bugs before release

3. Swagger / OpenAPI Documentation

- Add springdoc-openapi dependency
- Annotate all controllers with @Operation, @ApiResponse, @Parameter
- Group APIs by tag: auth, organizations, properties, floors, spaces, files, checklists, equipment, cable-paths, rf-scans, reports, audit
- Swagger UI available at /swagger-ui.html

4. Developer & User Documentation

- README.md — full local setup, environment variables, Docker Compose commands
- ARCHITECTURE.md — system design, DB diagram reference, tech choices
- USER_GUIDE_ADMIN.md — admin workflows with screenshots
- USER_GUIDE_ENGINEER.md — field engineer app walkthrough
- RELEASE_NOTES.md — list of all features, known limitations, version

5. Frontend Final Pass

- React routing: login, dashboard, property view, floor plan canvas, checklists, reports

- Tailwind responsive design — works on tablet (engineers use tablets on-site)
- Error handling: API errors shown as toast messages
- Loading states and form validation on all inputs

✅ **OUTCOME:** App is live on staging. All 3 roles tested end-to-end. Swagger docs accessible. README enables any developer to run the project in under 10 minutes.

Project Summary — All 14 Milestones

#	Milestone	Duration	Key Deliverable
1	Core Setup – Environment, DB & Project Skeleton	Week 1	Spring Boot + MySQL + Docker running
2	Authentication, JWT & Role-Based Access	Week 2	Login / refresh tokens + 3 roles secured
3	Organization & User Management	Week 2	Org creation, user CRUD, role assignment
4	Property Hierarchy (Property→Building→Floor→Space)	Week 3	Full location hierarchy APIs + DB
5	Floor Plan Upload & File Storage	Week 4	Image/PDF upload, MinIO/local, metadata in DB
6	Bulk Import – CSV / XLSX Parser	Week 4	Bulk space & building import with preview
7	Canvas Labeling & Space Geometry	Week 5	MapLibre canvas tagging saved as WKT
8	Checklist Templates & Survey Responses	Weeks 5-6	Templates created; engineers submit responses
9	Equipment & Cable Path Management	Week 6	Equipment inventory + cable routes stored
10	Attachments & File Linking	Week 6	Photos/files linked to any entity
11	RF Scan Integration (Vistumbler/Kismet/SPLAT!)	Week 7	RF data imported + heatmap overlays
12	Dashboards & Reports (PDF)	Week 7	Summary dashboards + PDF report generation
13	Audit Logs, Permissions & Testing	Week 8	Full audit trail + E2E tests passing
14	UAT, Staging Deploy & Documentation	Week 8	Staging live, Swagger docs, release notes

Total: 8 Weeks | 14 Milestones | 17 DB Tables | 3 Roles | Spring Boot + React.js + MySQL